

تقرير الفحص الأمني للموقع التابع لمديرية كربلاء المقدسة .

بناءً على الفحص الأخير للنظام الذي تم إجراءه يوم الأحد المصادف 5/3، تم استخلاص الثغرات الأمنية التالية مع تفصيل دقيق للأوامر المستخدمة، النتائج المترتبة، والتوصيات الهندسية.

الثغرة (1): كشف إعدادات ومفاتيح قاعدة البيانات (Supabase Information Exposure)

الوصف: تم اكتشاف تسريب لمفاتيح الربط الخاصة بقاعدة البيانات (Supabase Keys) وعناوين URL الخاصة بالخدمة ضمن ملفات JavaScript الخاصة بالتطبيق. هذا التسريب يتيح للمهاجمين المحتملين الوصول المباشر إلى نقطة نهاية API الخاصة بقاعدة البيانات والمفاتيح التي يمكن استخدامها للتفاعل معها.

الأمر المستخدم:

Bash

```
grep -rh "supabase\.co\|supabase\.com\|supabase" . --include="*.js" |
grep -oP 'https?://[a-zA-Z0-9.-]+supabase\.co[^\s"' ]*' | sort -u
grep -rh "eyJ" . --include="*.js" | grep -oP 'eyJ[a-zA-Z0-9_-]{10,}\. [a-zA-Z0-9_-]{10,}\. [a-zA-Z0-9_-]{10,}' | sort -u
grep -rh "service_role\|anon\|apikey\|api_key\|supabaseKey\|supabase_key" . --include="*.js" | head -20
```

النتيجة:

- اكتشاف URL قاعدة البيانات :
[https://jvnjkgxpnhrldlbczkgw.supabase.co] (https://jvnjkgxpnhrldlbczkgw.supabase.co)
- اكتشاف مفاتيح API: تم العثور على مفاتيح anon (مفاتيح الوصول العامة) ضمن الكود.
- إمكانية الوصول المباشر: باستخدام هذه المفاتيح، يمكن للمهاجمين محاولة الاستعلام عن البيانات أو التلاعب بها إذا لم يتم تطبيق سياسات أمان صارمة على مستوى الصفوف (Row Level Security - RLS).

التوصيات الهندسية:

1. **تفعيل Row Level Security (RLS):** يجب تفعيل RLS على جميع الجداول في قاعدة بيانات Supabase. هذا يضمن أن المستخدمين يمكنهم فقط الوصول إلى البيانات التي يمتلكون صلاحية الوصول إليها، حتى لو تمكن المهاجم من الحصول على مفتاح anon.
2. **إزالة المفاتيح الحساسة من الواجهة الأمامية:** يجب عدم تضمين مفاتيح API الحساسة (مثل service_role أو أي مفاتيح ذات صلاحيات واسعة) مباشرة في كود الواجهة الأمامية (Client-side code). إذا كانت هناك حاجة لمفاتيح anon، فيجب أن تكون صلاحياتها محدودة للغاية.
3. **استخدام متغيرات البيئة الآمنة:** يجب تخزين المفاتيح الحساسة في متغيرات بيئة آمنة على الخادم (Backend) أو في خدمات إدارة الأسرار (Secret Management Services) وعدم كشفها للعميل.

4. **تشفير الكود: (Obfuscation)** على الرغم من أنه ليس حلاً أمنياً كاملاً، إلا أن تشفير الكود البرمجي (JavaScript Obfuscation) يمكن أن يزيد من صعوبة استخراج المفاتيح والروابط من قبل المهاجمين.

الثغرة (2): كشف المسارات الإدارية عبر التخمين (Directory Brute-forcing)

الوصف: تمكنا من اكتشاف مسارات إدارية وحساسة على الموقع باستخدام أداة تخمين المسارات (Fuzzing)، مما يشير إلى ضعف في إخفاء أو حماية هذه المسارات.

الأمر المستخدم:

Bash

```
ffuf -u https://itpc-hr.vercel.app/FUZZ \
-w <(echo -e "login\n \ndashboard\n \nadmin-panel\nadmin\nemployees\nstaff\nleave\nleaves \nrequests\nattendance \ndocuments\nchat \nmessages \nsettings \nreports\napi\napi/v1\napi/users\napi/auth\napi/employees\napi/leaves\napi/requests\napi/admin\nauth/login\nauth/register\nauth/reset-password\nreset-password\nlogout\nwithdraw\nconfirm\nverify" ) \
-t 20 -c -s -r
```

النتيجة:

- **اكتشاف مسارات متعددة:** تم اكتشاف العديد من المسارات التي تعطي استجابة 200 OK، مما يعني أنها موجودة ويمكن الوصول إليها. من هذه المسارات:
 - / الموظفين
 - dashboard / لوحة التحكم
 - / مدير
 - / أدمن
 - / إجازات
 - login / تسجيل الدخول
 - / تسجيل الدخول - عربي
 - وغيرها الكثير من المسارات المتعلقة بالإدارة والموظفين.
- **مؤشر على نظام توجيه كامل باللغة العربية:** يشير هذا إلى أن التطبيق يستخدم نظام توجيه (Routing) يدعم المسارات باللغة العربية بشكل كامل.

التوصيات الهندسية:

1. **تطبيق آليات المصادقة والتفويض الصارمة:** يجب أن تكون جميع المسارات الإدارية والحساسة محمية بآليات مصادقة (Authentication) وتفويض (Authorization) قوية. لا يكفي مجرد إخفاء الرابط.
2. **التحقق من الصلاحيات على مستوى الخادم:** يجب أن يتم التحقق من صلاحيات المستخدم على مستوى الخادم (Backend) قبل السماح بالوصول إلى أي مورد أو وظيفة إدارية.
3. **تغيير أسماء المسارات الحساسة:** يفضل استخدام أسماء مسارات غير متوقعة أو معقدة للمناطق الإدارية، بدلاً من الأسماء الشائعة أو البديهية.
4. **استخدام Web Application Firewall (WAF):** يمكن لـ WAF المساعدة في اكتشاف ومنع هجمات تخمين المسارات (Brute-forcing) عن طريق تحليل أنماط الطلبات وحظر الأنشطة المشبوهة.

الثغرة (3): تسريب ملفات الخرائط المصدرية (Source Maps Leakage)

الوصف: يمكننا من الوصول إلى ملفات sourcemaps الخاصة بالتطبيق المستضاف على Vercel. هذه الملفات تسمح بتحويل الكود المجمع أو المصغر (Minified/Bundled code) إلى الكود المصدري الأصلي، مما يكشف عن بنية التطبيق الداخلية ومنطق العمليات، وهو ما يمثل خطرًا أمنيًا كبيرًا.

الأمر المستخدم:

Bash

```
curl -sI https://itpc-hr.vercel.app-v 2>&1 | grep -iE "vercel|x-vercel|x-env|etag|x-robots|x-frame|set-cookie"
curl -s https://itpc-hr.vercel.app/_vercel/insights
curl -s https://itpc-hr.vercel.app/_vercel/sourcemaps
curl -s https://itpc-hr.vercel.app/api/__vercel/env
```

النتيجة:

- الوصول إلى `_vercel/sourcemaps` أظهرت الأوامر أن هذا المسار قد يكون متاحًا.
- كشف الكود المصدري: يؤدي الوصول إلى ملفات sourcemaps إلى كشف الكود المصدري الأصلي للتطبيق، بما في ذلك منطق العمليات، أسماء المتغيرات، وهياكل البيانات، مما يسهل على المهاجمين فهم كيفية عمل التطبيق واكتشاف ثغرات أخرى.
- كشف معلومات البيئة: قد تكشف مسارات مثل `api/__vercel/env` عن متغيرات البيئة التي قد تحتوي على معلومات حساسة.

التوصيات الهندسية:

1. تعطيل أو إزالة Source Maps في بيئة الإنتاج: يجب التأكد من أن ملفات sourcemaps لا يتم توليدها أو رفعها إلى بيئة الإنتاج (Production Environment) على Vercel أو أي منصة استضافة أخرى. يمكن الاحتفاظ بها لبيئات التطوير (Development) أو الاختبار (Staging) فقط.
2. مراجعة إعدادات البناء (Build Configuration): التحقق من إعدادات أدوات البناء مثل (Webpack, Rollup) لضمان تعطيل توليد sourcemaps للـ Production builds.
3. حماية مسارات Vercel الخاصة: التأكد من أن المسارات الخاصة بمنصة Vercel مثل `/_vercel/*` و `api/__vercel/*` محمية بشكل صحيح ولا تكشف عن معلومات حساسة.

الثغرة (4): ضعف حماية الوظائف البرمجية الطرفية (Edge Functions Exposure)

الوصف: تم اكتشاف نقاط نهاية (Endpoints) لوظائف برمجية طرفية (Edge Functions) تتعلق بميزات حساسة مثل التحقق بخطوتين (2FA) وإرسال رسائل البريد الإلكتروني. هذه النقاط قد تكون عرضة للاستغلال إذا لم يتم تطبيق قيود كافية عليها.

الأمر المستخدم:

Bash

```
grep -rh "supabase\.co\|supabase\.com\|supabase" . --include="*.js" |  
grep -oP 'https?://[a-zA-Z0-9.-]+supabase\.co[^\s"' ]*' | sort -u
```

النتيجة:

- **اكتشاف نقاط نهاية 2FA:** تم العثور على روابط مثل
[https://jvnjkqxpnhridlbczkkgw.supabase.co/functions/v1/auth-verify-2fa] (https://jvnjkqxpnhridlbczkkgw.supabase.co/functions/v1/auth-verify-2fa)
[https://jvnjkqxpnhridlbczkkgw.supabase.co/functions/v1/send-2fa-email] (https://jvnjkqxpnhridlbczkkgw.supabase.co/functions/v1/send-2fa-email).
- **إمكانية إساءة الاستخدام:** يمكن للمهاجمين استغلال هذه النقاط لمحاولة تخمين رموز 2FA، أو إرسال رسائل بريد إلكتروني مزعجة (Spam)، أو شن هجمات حجب الخدمة (DoS) إذا لم تكن هناك قيود على معدل الطلبات (Rate Limiting).

التوصيات الهندسية:

1. **تطبيق Rate Limiting:** يجب تطبيق قيود صارمة على معدل الطلبات (Rate Limiting) على جميع نقاط نهاية API، وخاصة تلك المتعلقة بالمصادقة، تسجيل الدخول، وإرسال الإشعارات، لمنع هجمات التخمين وحجب الخدمة.
2. **التحقق من صحة المدخلات (Input Validation):** يجب التحقق بدقة من جميع المدخلات التي تستقبلها هذه الوظائف لضمان أنها تتوافق مع التوقعات المتوقعة ومنع حقن الأكواد الضارة.
3. **استخدام Captcha أو آليات التحقق البشري:** يمكن إضافة آليات مثل reCAPTCHA أو تحديات أخرى للتحقق من أن الطلبات لا تأتي من برامج آلية.
4. **مراقبة السجلات والإنذارات:** يجب مراقبة سجلات الوصول لهذه الوظائف بشكل مستمر وإعداد تنبيهات لأي نشاط مشبوه أو عدد غير طبيعي من الطلبات.

الجانب الأمني	التوصية الهندسية المفصلة
حماية البيانات	تفعيل Row Level Security (RLS) بشكل كامل: يجب تطبيق RLS على جميع الجداول في قاعدة بيانات Supabase لضمان أن كل مستخدم يمكنه فقط الوصول إلى البيانات التي يمتلك صلاحية الوصول إليها، حتى لو تمكن المهاجم من الحصول على مفتاح anon. يجب أن تكون سياسات RLS محكمة ومراجعة بانتظام.
إدارة الأسرار	تخزين المفاتيح الحساسة بشكل آمن: يجب إزالة جميع المفاتيح الحساسة (مثل service_role أو أي مفاتيح ذات صلاحيات واسعة) من كود الواجهة الأمامية. يجب تخزينها في متغيرات بيئة على الخادم أو في خدمات إدارة الأسرار المخصصة (مثل AWS Secrets Manager, Google Secret Manager) والوصول إليها فقط من بيئة الخادم.
إخفاء المعلومات	تعطيل Source Maps في الإنتاج: يجب تعطيل توليد أو رفع ملفات sourcemaps في بيئة الإنتاج لمنع كشف الكود المصدري الأصلي للتطبيق. يمكن الاحتفاظ بها لبيئات التطوير والاختبار فقط.
التحقق من الصلاحيات	تطبيق المصادقة والتفويض على مستوى الخادم: يجب أن تكون جميع المسارات الإدارية والحساسة محمية بآليات مصادقة وتفويض قوية. يجب التحقق من صلاحيات المستخدم على مستوى الخادم (Backend) قبل السماح بالوصول إلى أي مورد أو وظيفة إدارية، وليس فقط الاعتماد على إخفاء الروابط.
مكافحة هجمات التخمين	تطبيق Rate Limiting: يجب تطبيق قيود صارمة على معدل الطلبات (Rate Limiting) على جميع نقاط نهاية API، وخاصة تلك المتعلقة بالمصادقة، تسجيل الدخول، وإرسال الإشعارات، لمنع هجمات التخمين وحجب الخدمة.
أمان الكود	تشفير الكود (Obfuscation) للواجهة الأمامية: على الرغم من أنه ليس حلاً أمنياً كاملاً، إلا أن تشفير الكود البرمجي للواجهة الأمامية يمكن أن يزيد من صعوبة استخراج المفاتيح والروابط ومنطق العمليات من قبل المهاجمين.
مراقبة الأمان	مراقبة السجلات والإنذارات: يجب تفعيل المراقبة المستمرة لسجلات الوصول ونشاط المستخدمين على جميع نقاط النهاية الحساسة، وإعداد تنبيهات فورية لأي أنشطة مشبوهة أو أنماط وصول غير طبيعية.